Démarrage et arrêt du système Linux

- Introduction
- Processus de démarrage
- Démarrage: BIOS
- Chargeur de démarrage: BootLoader
- Démarrage du noyau
- Lancement du processus PID 1
- Gestionnaires de services
- Arrêt du système
- Gestion de services
- Dépannage et journalisation

Démarrage et arrêt du système Linux

Introduction

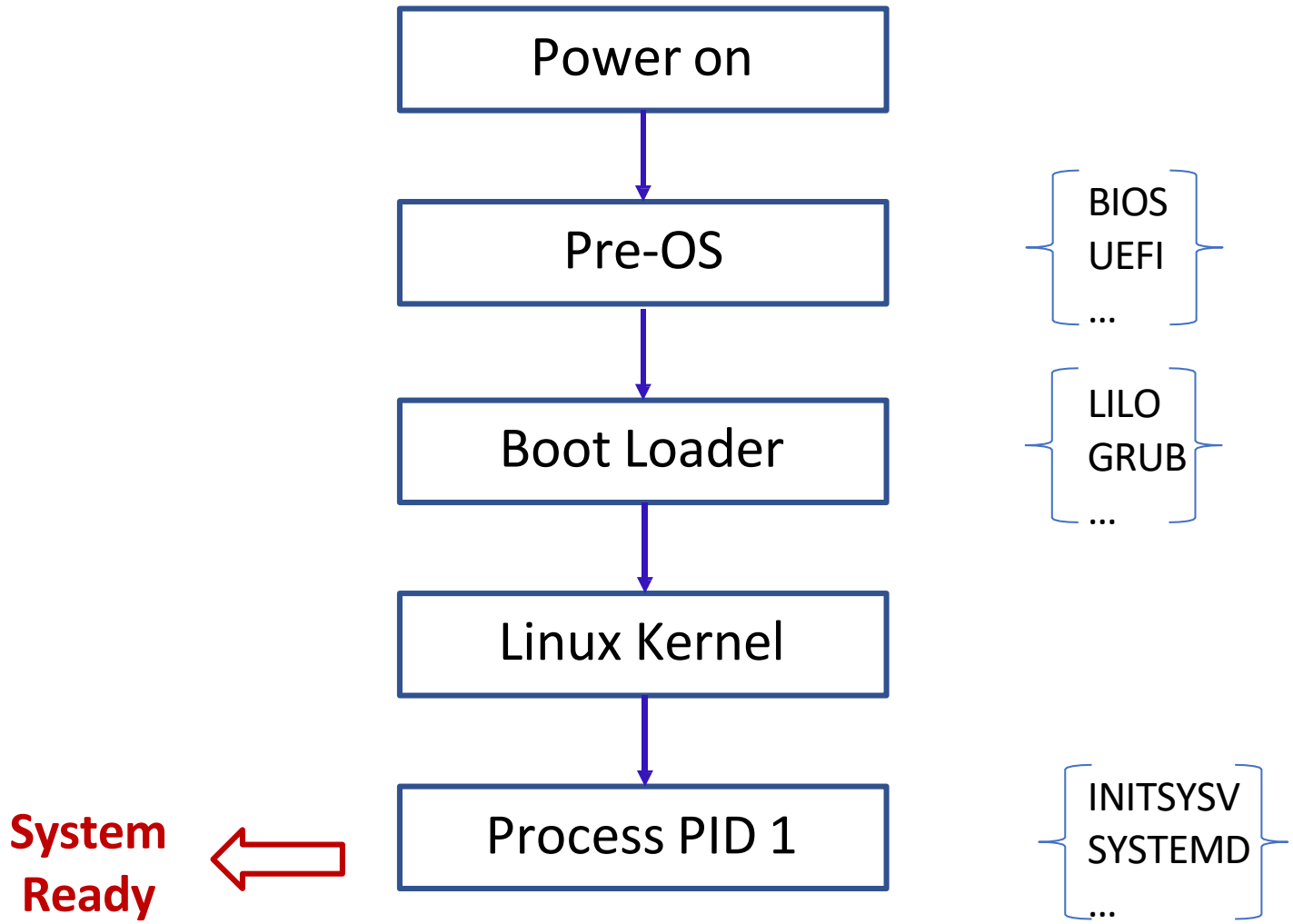
- Le système informatique subit plusieurs phases de **processus d'amorçage** (« boot strap process ») depuis l'événement de mise sous tension jusqu'à ce qu'il offre à l'utilisateur un système d'exploitation (OS) **pleinement fonctionnel**.

  ⇒ Pour qu'un système GNU/Linux soit utilisable, il doit d'abord passer par **plusieurs étapes d'initialisation** et de lancement de divers programmes.

  ⇒ Lorsque l'ordinateur démarre, les messages défilant sur la console révèlent de nombreuses **initialisations et configurations** automatiques. Parfois, il est souhaitable de **modifier légèrement le déroulement de cette étape**, ce qui implique de bien la comprendre.

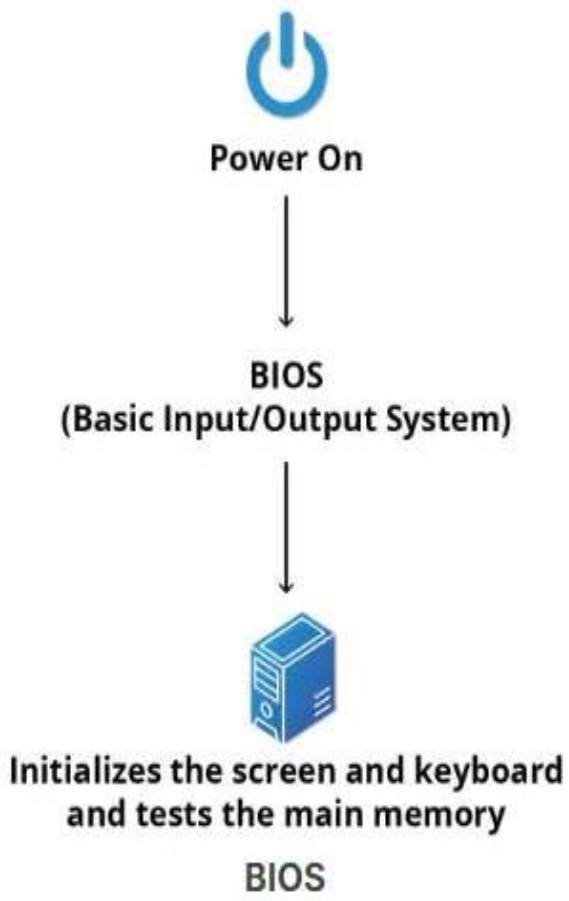
Démarrage et arrêt du système Linux

Processus de démarrage



Démarrage et arrêt du système Linux

Démarrage: BIOS (1)



- **BIOS:** Basic Input Output System
- Se situe sur **la ROM** de la carte mère
- Le BIOS fait quelques tests (**POST : Power-On Self Tests**) sur le matériel. A ce stade, on peut arrêter le BIOS pour le configurer (selon les bios, ex : touche DEL).
- **Analyse la configuration matérielle** de l'ordinateur: Recensement des périphériques + Test de certains périphériques :
  - ⇒ Test de la mémoire,
  - ⇒ Test de la présence de clavier, ...
  - ⇒ Test de la présence des disques durs, lecteurs de CDROM ...
- **Localisation du MBR :** il cherche le MBR dans le premier secteur (secteur 0) du périphérique de démarrage sélectionné (disque dur, clé USB, etc.), tel que défini par l'ordre de démarrage.

Démarrage et arrêt du système Linux

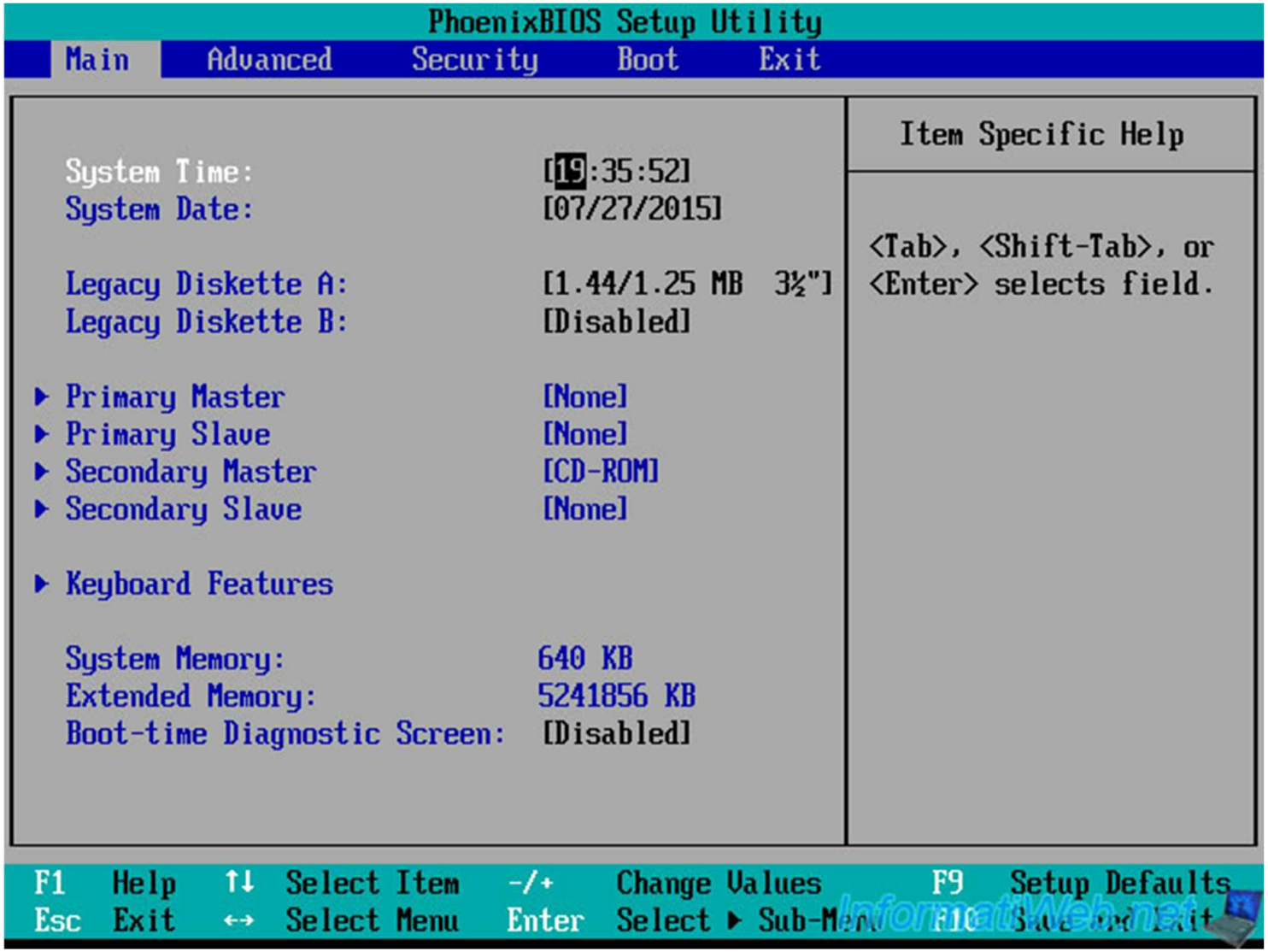
Démarrage: BIOS (2)





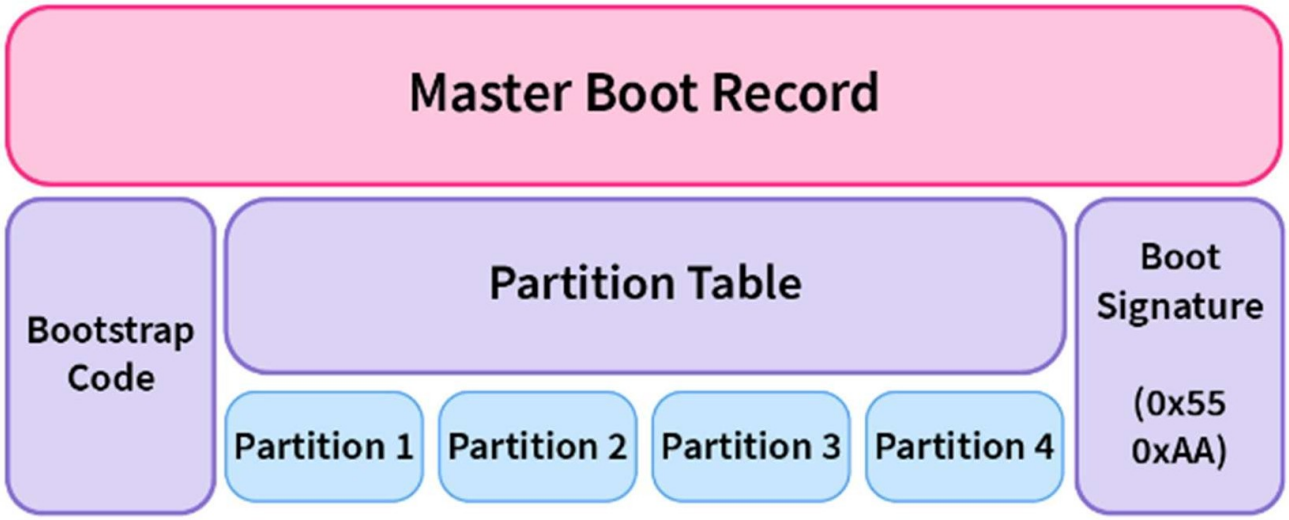
Démarrage et arrêt du système Linux

Démarrage: BIOS (3)



Démarrage: BIOS (5)

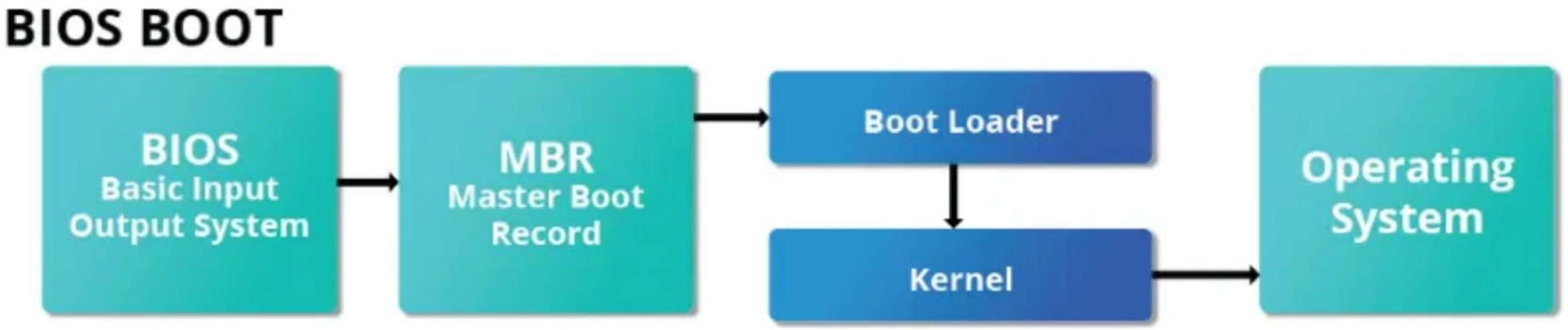
- Localisation de MBR:
  - ⇒ lit les 512 premiers octets du disque dur:
    Ces 512 octets constituent le **MBR = Master Boot Record**
- Ce secteur contient:
  - ⇒ **un code d'amorçage (Bootloader de l'étape 1)**
  - ⇒ **la table des partitions**



Démarrage et arrêt du système Linux

Démarrage: BIOS/MBR

1. **Le BIOS** du système examine le système et lance **le chargeur de démarrage** de **l'Étape 1 (code d'amorçage)** sur le bloc de démarrage maître (MBR) du disque dur principal.
2. **Le chargeur de démarrage** de **l'Étape 1** se charge en mémoire et lance **le chargeur de démarrage** de **l'Étape 2** à partir de la partition **/boot/. (Ex: Grub)**



Démarrage et arrêt du système Linux

Démarrage: UEFI (1)

- **UEFI: U**nified **E**xtensible **F**irmware **I**nterface, la dernière méthode de démarrage d'un ordinateur conçue pour **remplacer le BIOS.**
- **UEFI** est généralement utilisé sur les systèmes 64 bits ultérieurs à Windows 7
- la plupart des nouvelles cartes mères permettent également aux utilisateurs de passer en mode de compatibilité **Legacy + UEFI**.
- En mode de compatibilité **Legacy + UEFI**, le firmware peut démarrer un système en mode **Legacy** ou en mode **UEFI**, selon le mécanisme d'amorçage disponible et l'entrée de démarrage sélectionnée.
- Le système d'exploitation doit disposer des composants de démarrage compatibles avec le mode choisi.

Démarrage: UEFI (2)

BIOS vs UEFI

- Le **UEFI** est un programme qui est conçu pour **remplacer le BIOS**
- Il tient plus du système d'exploitation que du BIOS:
  - ⇒ Interface graphique à fenêtre
  - ⇒ Accès à Internet
  - ⇒ Mesures de sécurité et anti-virus intégrés
  - ⇒ Support **GPT** (GUID Partition Table—remplacement du MBR) pour démarrer sur des partitions de plus de 2TB
  - ⇒ Architecture modulaire et une grande partie est **écrite en C** au lieu d'en assembleur ce qui rend l'adaptation pour d'autres plateformes plus facile

Démarrage et arrêt du système Linux

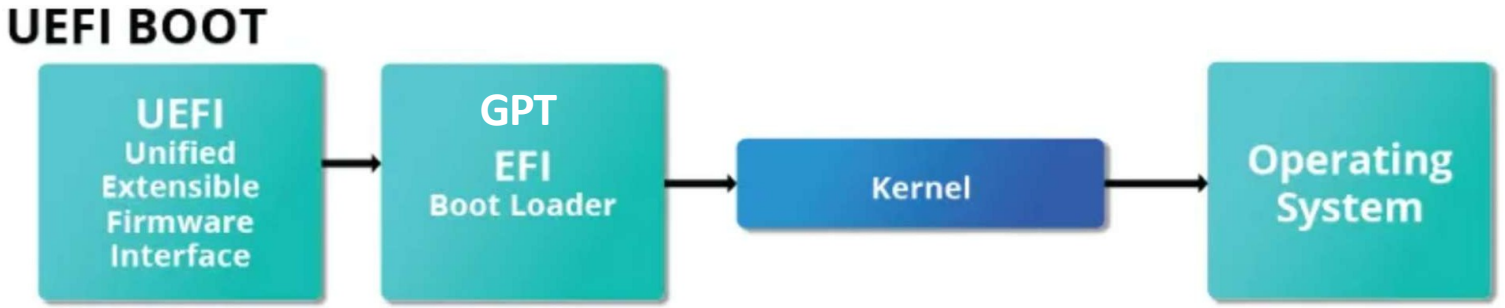
Démarrage: UEFI (3)



Démarrage et arrêt du système Linux

Démarrage: UEFI/GPT

1. **UEFI** lit la table des partitions **(GPT)** sur le stockage du système et monte l'**ESP (EFI System Partition).** L'**ESP** contient **des application EFI**, comme des **chargeurs de démarrage** et logiciels d'utilitaires stockés dans des répertoires spécifiques aux fournisseurs de logiciels. A partir du système **Red Hat Enterprise Linux 6**, l'**ESP** est **/boot/efi/.**
2. Le répertoire **/boot/efi/EFI/redhat/** contient **grub.efi,** une version de **GRUB** compilée pour les systèmes qui utilisent **UEFI**. Dans le cas le plus simple, le gestionnaire de démarrage EFI sélectionne **grub.efi** comme **chargeur de démarrage** et le charge dans la mémoire.



Démarrage et arrêt du système Linux

Chargeur de démarrage (Boot Loader) (1)

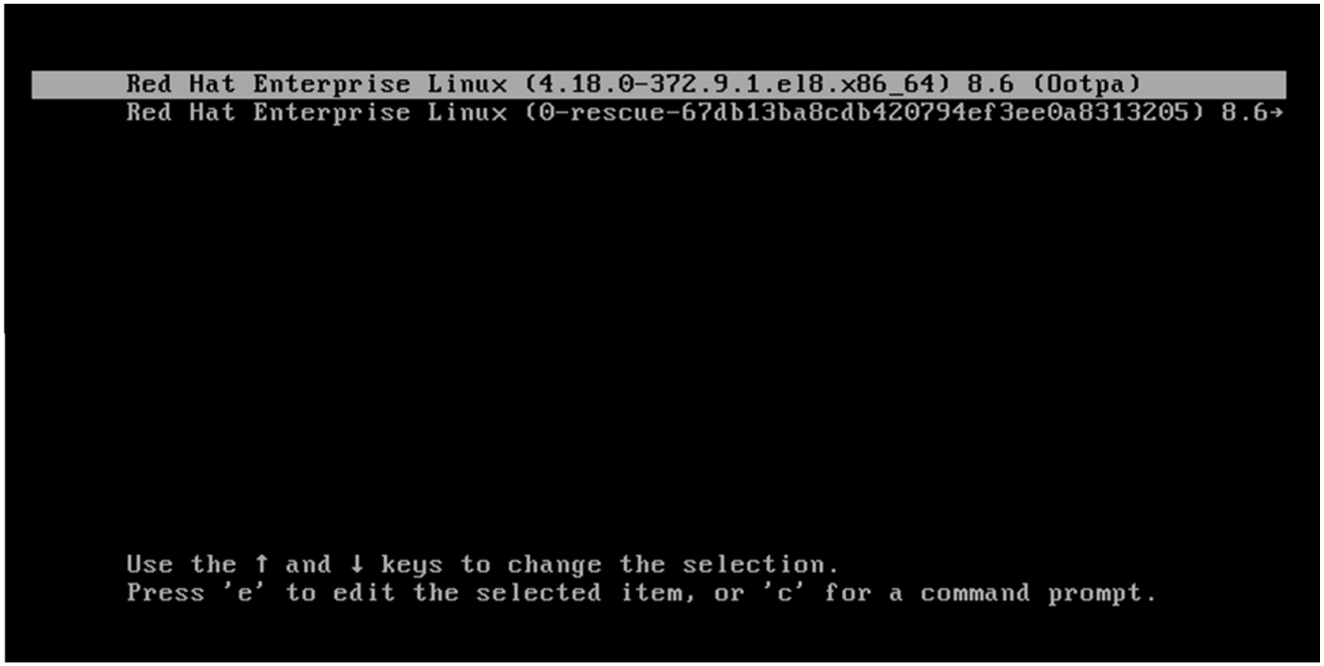
- Pour qu'un système Unix puisse se lancer, il faut que **le noyau soit chargé en mémoire et qu'il s'exécute**
- Le but principal d'un **boot loader** est de charger le noyau en mémoire et de le lancer
- Un bootloader peut permettre, aussi, de
  - ⇒ Choisir entre plusieurs noyaux à charger
  - ⇒ Passer des paramètres au noyau chargé
  - ⇒ Choisir entre plusieurs OS (sur un système multi OS)
- Le boot loader **est obligatoire** pour charger le noyau
- Le boot loader fait la transition entre le démarrage matériel de la machine (**mise sous tension**) et l'exécution du noyau (**lancement de l'OS**)

Démarrage et arrêt du système Linux

Chargeur de démarrage (Boot Loader) (3)

- Le chargeur de démarrage **localise le noyau** du système d'exploitation sur le disque, le charge et l'exécute
- Lorsque l' ordinateur héberge **plusieurs systèmes** (multi-amorçage), il permet à l'utilisateur de **choisir quel système démarrer.**

- Types de chargeurs:
  - ⇒ **Grub** : GRand Unified Bootloader (Première version)
  - ⇒ **Grub2** : nouvelle version
  - ⇒ **Lilo** : LInux LOader (délaissé par les développeurs) ;



Chargeur de démarrage (Boot Loader) (4)

Mise en place de GRUB 2

- La plupart des distributions récentes **installent automatiquement** GRUB2.
- Cette étape n'est alors nécessaire que pour réinstaller GRUB2 ou migrer de **Grub Legacy** vers **Grub2** dans le cas d'une ancienne distribution.
- La procédure de mise en place comprend :
  - ⇒ L'installation ou mise à jour des paquetages
  - ⇒ L'installation du chargeur
  - ⇒ Configuration

Mise en place de GRUB 2

- L'installation des paquetages peut se faire à l'aide de la commande :

```
yum -y install grub2 os-prober
```

ou

```
yum -y update grub2 os-prober
```

pour la mise à jour

- Le paquetage **os-prober** est utilisé particulièrement par **grub2** pour chercher les systèmes installés et générer les entrées correspondantes dans le fichier **grub.cfg**

Démarrage et arrêt du système Linux

Chargeur de démarrage (Boot Loader) (5)

- **Fichiers Grub2:**

  ⇒ **/etc/default/grub** - le fichier contenant les **paramètres du menu** de GRUB 2,

  ⇒ **/etc/grub.d/** - le répertoire contenant **les scripts de création du menu GRUB 2**, permettant notamment de personnaliser le menu de démarrage,

  ⇒ **/boot/grub2/grub.cfg** - le **fichier de configuration final** de GRUB 2, **non modifiable**. **(/boot/grub/grub.cfg** sous Debian).

  ⇒ Le dernier fichier est généré automatiquement par le programme **grub2-mkconfig** à partir des scripts **/etc/default/grub** et **/etc/grub.d/**

Chargeur de démarrage (Boot Loader) (6)

Configuration de GRUB 2

- Avec **grub2** on n'édite pas directement le fichier **grub.cfg**, mais on intervient sur :
  - ⇒ **/etc/default/grub** : qui contient les valeurs de quelques paramètres, comme la résolution de l'écran, le Timeout, le système par défaut, ...
  - ⇒ **/etc/grub.d/*** : un ensemble de scripts permettant de générer le fichier **grub.cfg**. Et dans lesquels on peut rajouter par exemple le code permettant de mettre en place un mot de passe de démarrage.
- Après modification, la commande **grub2-mkconfig** doit être appelée pour régénérer un nouveau **grub.cfg**

Démarrage et arrêt du système Linux

Chargeur de démarrage (Boot Loader) (7)

Fichier /etc/default/grub

- Contient les valeurs associées aux **paramètres de démarrage**
- Chacune de ses lignes est sous forme : **VARIABLE=valeur**

Exemple:

```
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR="$(sed 's, release .*$,,g' /etc/system-
release)"
GRUB_DEFAULT=saved
GRUB_DISABLE_SUBMENU=true
GRUB_TERMINAL_OUTPUT="console"
GRUB_CMDLINE_LINUX="crashkernel=auto rd.lvm.lv=centos/root
rd.lvm.lv=centos/swap rhgb quiet"
GRUB_DISABLE_RECOVERY="true"
```

Chargeur de démarrage (Boot Loader) (8)

Fichier /etc/default/grub: quelques paramètres

- **GRUB_TIMEOUT : durée en seconde** à attendre avant le démarrage du système par défaut
- **GRUB_DEFAULT :** désigne **le système à démarrer par défaut**, sa valeur peut être l'ordre du système dans le menu (0 est le premier) son nom ou le mot clé **saved** pour désigner le dernier système activé
- **GRUB_DISTRIBUTOR**: Défini par les distributeurs de GRUB et utilisé pour générer des titres d'entrée de menu plus informatifs. Par exemple :

```
GRUB_DISTRIBUTOR=`echo -n TITRE PERSONNALISÉ`#donne : TITRE
PERSONNALISÉ GNU/Linux
```

Chargeur de démarrage (Boot Loader) (9)

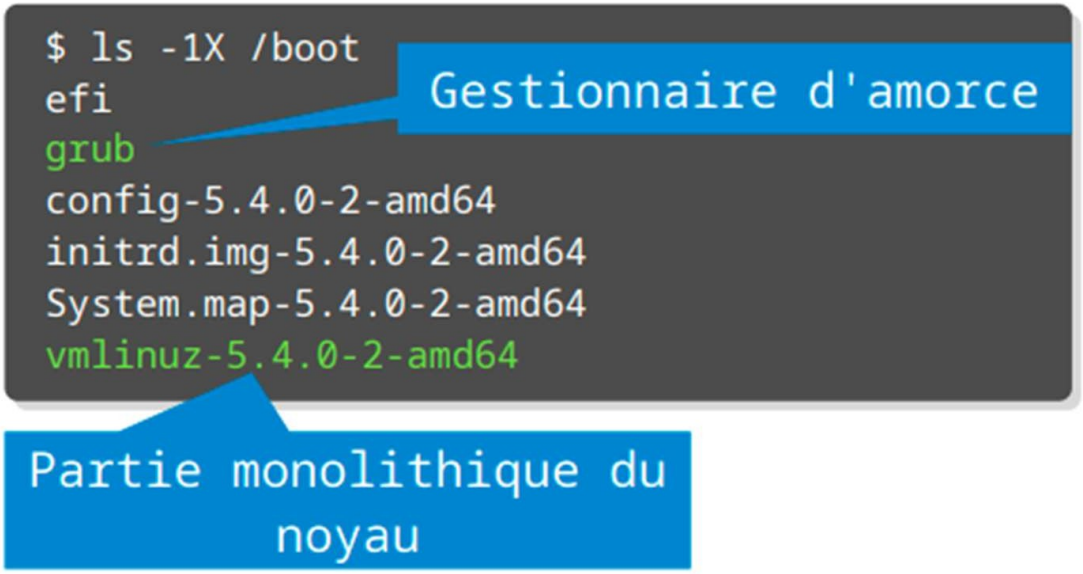
Fichier /etc/default/grub: quelques paramètres

- **GRUB_DISABLE_SUBMENU** : si on souhaite voir directement une entrée de menu par noyau linux disponible (On peut supprimer les noyaux les plus anciens → Nettoyer le système)
- **GRUB_CMDLINE_LINUX :** les arguments à passer au noyau Linux
- **GRUB_DISABLE_RECOVERY** : la valeurs **true** de ce paramètre permet de désactiver la génération de l'entrée du menu relative au mode **Recovery**
- **GRUB_BACKGROUND:** Paramètre à ajouter si vous voulez mettre une image en arrière plan.

Démarrage et arrêt du système Linux

Chargeur de démarrage

- **Le boot loader charge et exécute le noyau sélectionné** et l'image **initrd** ou **initramfs** = *INITial RAM filesystem*).
- Le fichier du noyau s'appelle en général **vmlinux** sur linux (vmlinuz est sa version compressée autoextractible).



/boot: fichiers de configuration du démarrage (boot), noyaux et d'autres fichiers indispensables au moment du démarrage (boot).

Démarrage et arrêt du système Linux

InitRamFs: Initial RAM Disk (1)

- Pour pouvoir démarrer les applications, il faut pouvoir lire les exécutables depuis **un système de fichiers**
- Le noyau doit:
  - ⇒ Localiser le **système de fichiers racine (/)** et le monter .
  - ⇒ Disposer **du/des pilote(s)** de périphérique nécessaire(s) pour accéder au système de fichiers
- Si tous les pilotes nécessaires ont été compilés en dur dans le noyau **== > Noyau très volumineux**
- Le plus souvent, **les pilotes** sont généralement compilés sous forme de modules (afin de pouvoir charger uniquement ceux **utiles** dans la configuration donnée)
- Or les modules sont des fichiers et pour pouvoir les charger, il faut que le système de fichiers sur lequel ils sont **soit lui-même monté...**

InitRamFs: Initial RAM Disk (2)

Utilité d'un système de fichiers initial

- Donc, dans de nombreuses distributions classiques, **les pilotes de périphérique nécessaires pour accéder au système de fichiers** racine ne sont pas compilés en dur
- Donc le noyau **ne peut pas monter ce système de fichiers racine...**
- La solution passe par une image de système de fichiers racine initial (**initrd/initramfs**)
- **L'image est chargée en mémoire** par le bootloader **et son adresse est passée au noyau**

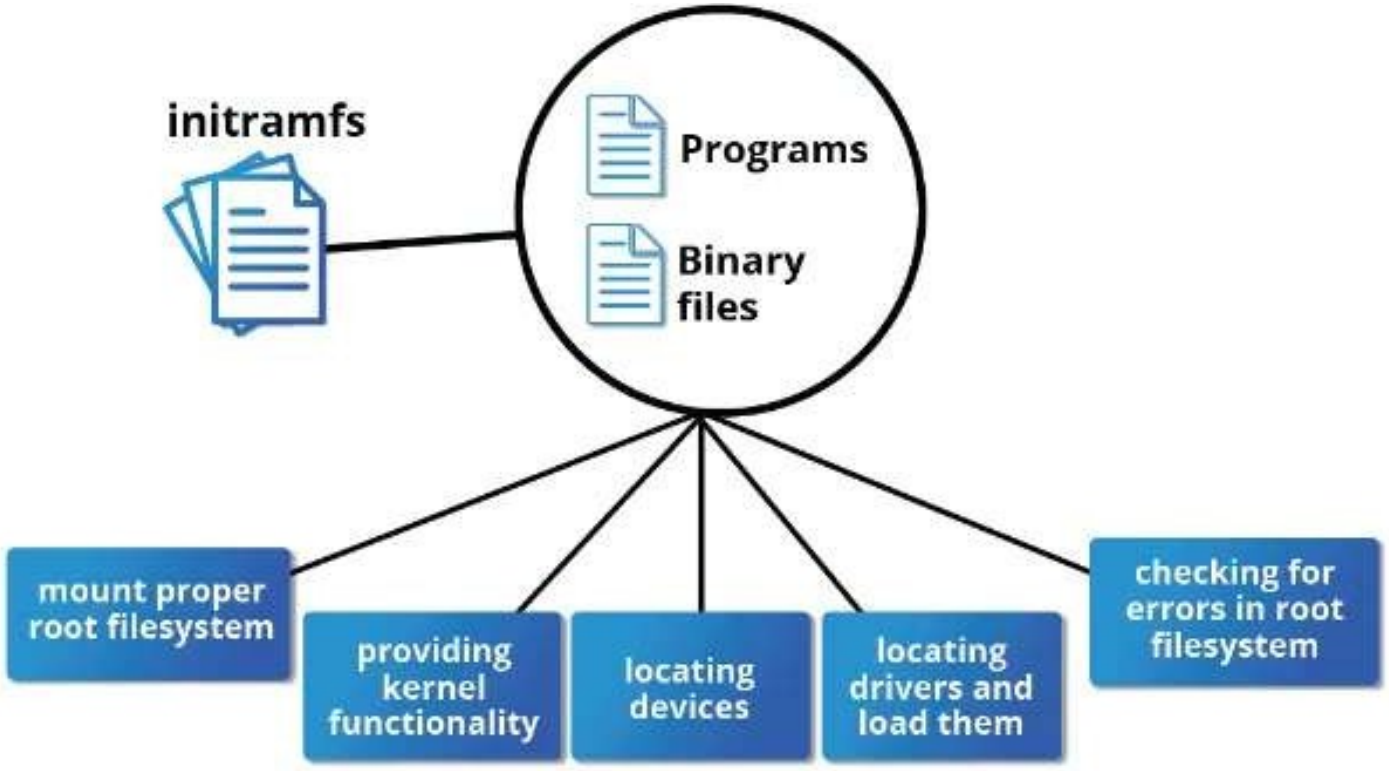
InitRamFs: Initial RAM Disk (3)

Initrd -- initramfs

- **Initramfs** est utilisé par le noyau comme système de fichiers racine **temporaire** jusqu'à ce que le noyau soit démarré et que le vrai système de fichiers racine soit monté. Il contient les pilotes nécessaires compilé à l'intérieur, ce qui l'aide à accéder aux partitions du disque dur et d'autres matériels
- **Initrd** est un ancien nom: actuellement **initramfs**
- Il contient **le strict minimum** qui peut être requis par le noyau pour charger le « **vrai** » système de fichiers racine :
  ⇒ il peut s'agir de modules de pilotes pour les disques durs ou d'autres périphériques **sans lesquels le système ne peut pas démarrer**, ou, plus fréquemment, des modules et des scripts d'initialisation permettant d'ouvrir des partitions chiffrées, …

Démarrage et arrêt du système Linux

InitRamFs: Initial RAM Disk (4)

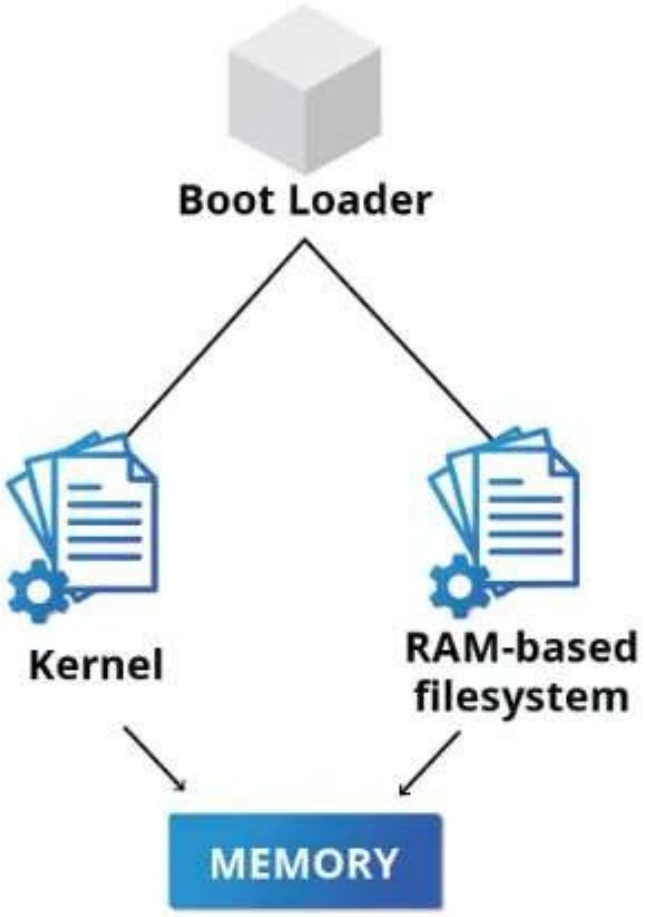


The Initial RAM Disk

Démarrage et arrêt du système Linux

Démarrage dy noyau (1)

- Au moment du démarrage, **le chargeur de démarrage** charge **le noyau** et l'image **initramfs** dans la mémoire et **démarre le noyau**.
- **Le noyau** vérifie la présence d'un **initramfs** et, s'il le trouve, le monte sur **/** et lance /init.



The Linux Kernel

Démarrage et arrêt du système Linux

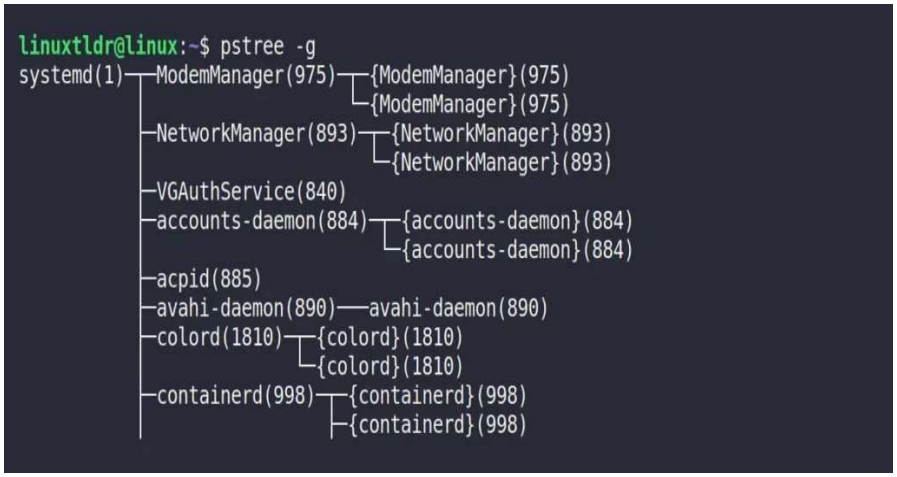
Lancement du Processus PID 1

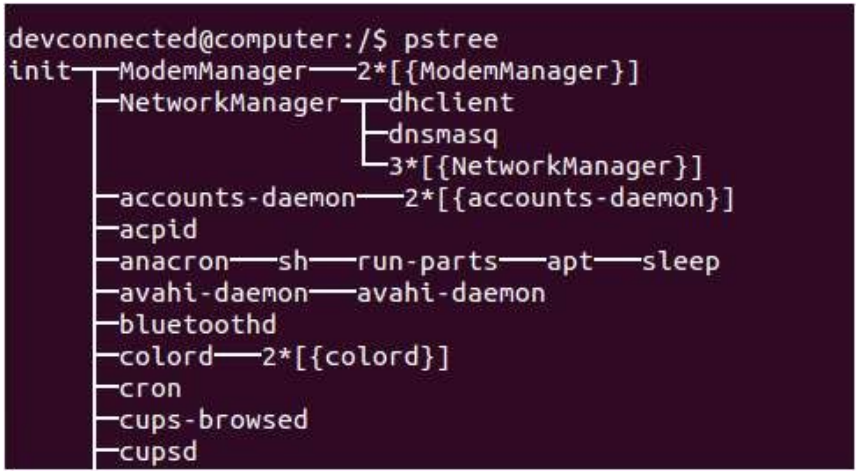
- A ce stade, le noyau **libère l'espace utilisé précédemment** pour la détection et la configuration des périphériques
- Ensuite, il lance le premier processus **(PID 1)**
- Sur un système UNIX, **le tout premier processus (PID 1)** a un rôle particulier :
  - ⇒ **c'est le seul qui est lancé par le noyau** ; il doit donc assurer toute l'initialisation du système, le lancement des différents services ;
  - ⇒ Le noyau **panique** s'il n'est pas capable de le trouver (**Kernel panic - not syncing: No working init found**.)

Démarrage et arrêt du système Linux

Gestionnaires de système et de services

- Le processus **PID 1** est donc le processus d'initialisation du système et de gestion des services
- Différents **systèmes de gestion des services** sont utilisés :
  - ⇒ **SysVinit:** processus init historique, hérité de UNIX System V.
  - ⇒ **Upstart:** sorti en 2006, à l'origine pour Ubuntu.
  - ⇒ **Systemd:** sorti en 2010.
- La commande : **pstree** permet d'afficher l'arbre des processus. Celui au sommet est **le gestionnaire de services utilisé**

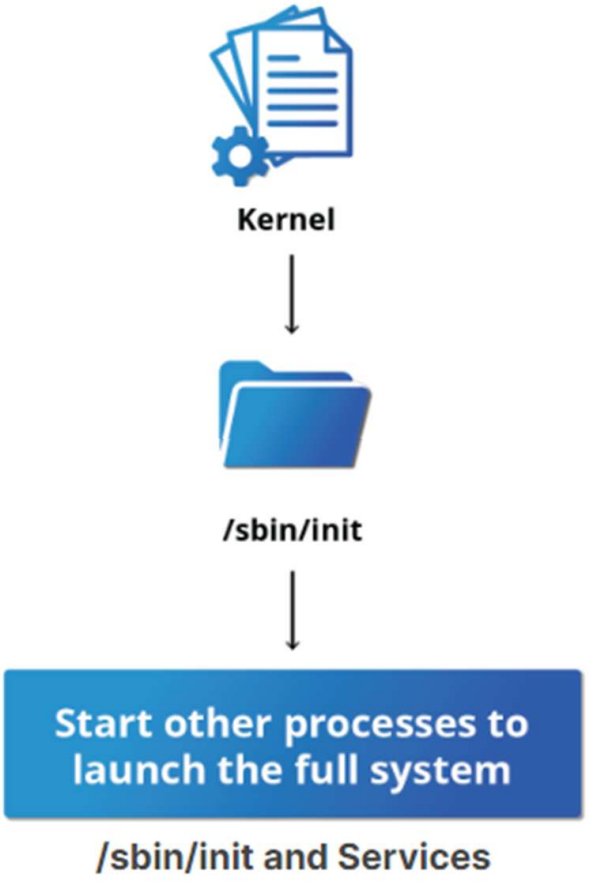




Démarrage et arrêt du système Linux

Gestionnaire de service SysVinit

- **SysVinit** est le système d'initialisation hérité d'UNIX Système V
- Ce système est représenté par le processus **/sbin/init**
- Le processus **/sbin/init** a le **pid = 1**
- Il est le parent de tous les processus présents sur le système
- Le processus **init** effectue un contrôle des partitions puis procède au montage du système de fichier principal sous **/**
- Après, **init** lance les différents services suivant le contenu du fichier **/etc/inittab** en faisant la distinction entre les différents **niveaux d'exécution (runlevels)**



Niveaux d'exécution (1)

- Le fonctionnement d'un système Linux régi par un ensemble de **niveaux d'exécution**
- La gestion des niveaux d'exécution consiste à déterminer quel doit être **le comportement du système** quand il entre dans un niveau donné.

Les niveaux d'exécution représentent différents modes dans lesquels un ordinateur peut fonctionner. Chaque niveau d'exécution définit un ensemble spécifique de services ou processus qui doivent être exécutés ou arrêtés lors du démarrage ou de l'arrêt du système.

Niveaux d'exécution (2)

- La distribution RedHat distingue les niveaux d'exécution suivants :

  ⇒ **Le niveau 0: arrêt:** correspond à l'arrêt du système, et aucun service n'est disponible.

  ⇒ **Le niveau 1,s,S: mode utilisateur unique:** réservé aux opérations de maintenance et ne permet qu'une seule connexion (compte root). La plus part des services sont arrêtés dans ce niveau (le système a une activité minimum). Parfait pour l'administrateur qui souhaite effectuer des opérations de maintenance.

  ⇒ **Le niveau 2: non utilisé- peut être défini par l'utilisateur**

Niveaux d'exécution (3)

⇒ **Le niveau 3: mode multi-utilisateurs complet:** Tous les services nécessaires sont démarrés et plusieurs utilisateurs peuvent se connecter **en mode texte (**l'interface graphique n'est pas disponible)

⇒ **Le niveau 4: non utilisé- peut être défini par l'utilisateur**

⇒ **Le niveau 5: mode multi-utilisateurs graphique:** Identique au mode 3, mais les utilisateurs peuvent se connecter en mode graphique et disposer d'un gestionnaire de fenêtre.

⇒ **Le niveau 6: redémarrage:** Passer dans ce niveau redémarre la machine. Après le démarrage, le système se trouve dans le mode indiqué par **initdefault** dans **/etc/inittab**.

Niveaux d'exécution: commandes

- La commande : **`init N`** ou **`telinit N`** permet de faire passer le système au niveau passé en argument
- La commande **`runlevel`** ou **`who -r`** permet d'afficher le niveau courant ainsi que le précédent

Fichier /etc/inittab

- Le comportement d'**init** est défini dans le fichier de configuration **/etc/inittab**
- Ce fichier contient :
  - ⇒ la description des **niveaux d'exécution**
  - ⇒ le **niveau par défaut** dans lequel le système se place au démarrage
  - ⇒ **les actions** qu'**init** doit effectuer lorsque certains évènements arrivent
- Il y est indiqué quels sont les scripts qui doivent être exécutés lors du **changement de niveau d'exécution**

Fichier /etc/inittab: structure

- Typiquement, une entrée dans le fichier **/etc/inittab** a la forme suivante :

  `id:niveaux_exécution:action:processus`

  ⇒ **Id:** Séquence unique de 1 à 4 caractères qui identifient une entrée dans inittab

  ⇒ **niveaux_exécution:** Liste des niveaux d'exécution pour lesquels l'action doit être faite.

  ⇒ **action:** Description de l'action à faire.

  ⇒ **processus:** Commande à exécuter.

Fichier /etc/inittab: Exemple

```
# The default runlevel.
:5:initdefault:

# Boot-time system
configuration/initialization
script.

l0:0:wait:/etc/init.d/rc 0
l1:1:wait:/etc/init.d/rc 1
l2:2:wait:/etc/init.d/rc 2
l3:3:wait:/etc/init.d/rc 3
l4:4:wait:/etc/init.d/rc 4
l5:5:wait:/etc/init.d/rc 5
l6:6:wait:/etc/init.d/rc 6

# What to do when CTRL-ALT-DEL is
pressed.
ca:12345:ctrlaltdel:/sbin/shutdown
-t1 -a -r now
```

- **Le niveau 5** est le niveau par défaut
- La combinaison **Controle-alt-del** lancera un "shutdown" immédiat puis de /etc/init.d/rc 2

Répertoires
/etc/rcX.d

- Le fichier **inittab** précédent nous indique qu'à l'entrée dans un niveau d'exécution donné **(ex. 3),** le script **/etc/rc.d/rc** est exécuté avec l'argument **3**
- Ce script va aller dans le répertoire **/etc/rc.d/rcx.d** (où x est remplacé par l'argument, ici 3)
- Dans ce répertoire, on trouve des fichiers (le plus souvent les liens symboliques) dont le nom est de la forme **Knnwwwwww** (où nn est un nombre et **wwwwww** un nom quelconque) ou **Snnwwwwww**
- Le plus souvent, ces fichiers sont des liens symboliques vers des scripts contenus dans **/etc/rc.d/init.d**
- Ces scripts portent **le nom du service qu'ils gèrent** et vont réagir à l'argument **start** ou **stop** en **démarrant/arrêtant le service concerné**

Démarrage et arrêt du système Linux

Gestionnaire de service SysVinit: Résumé

- Le système init:
  - ⇒ **Un niveau d'exécution** (runlevel) défini par **/etc/inittab**
  - ⇒ **Des scripts** dans **/etc/init.d** lançant les processus
  - ⇒ **Des répertoires /etc/rcx.d**, pour x = 0,1,...,6 contenant des liens symboliques vers les scripts de /etc/init.d



init est remplacé récemment: système vieillissant !!

Démarrage et arrêt du système Linux

Gestionnaire de service SysVinit: inconvénients

- Malheureusement, **SysVinit** reste très basique, peu performant et sujet à de nombreux problèmes. Il n'est pas très adapté à toutes les évolutions modernes que connaît Linux.
- Ces principaux défauts :
  - ⇒ Le démarrage des services est **séquentiel**.
  - ⇒ La prise en charge des **dépendances entre services** est très basique
  - ⇒ Tout ou presque est sous forme de scripts Shell ; ceci rend le processus de démarrage **lent** et **fastidieux**.
  - ⇒ ...
- Plusieurs alternatives ont été développées et **systemd** s'est progressivement imposé dans la majorité des distributions Linux généralistes ces dernières années

Démarrage et arrêt du système Linux

Gestionnaire de service Systemd

- **Systemd** est un gestionnaire de systèmes et de services pour les systèmes d'exploitation Linux. Il est conçu pour être rétro-compatible avec **les scripts init**, et fournit un certain nombre de fonctionnalités:

  ⇒ Peut lancer **en parallèle** des processus interdépendants et **diminuer le nombre de processus** lancés au démarrage **--> Accélérer le démarrage du système**

  ⇒ **Rationaliser** la gestion des services

  ⇒ **Uniformiser la gestion des services** (architecture commune à toutes les distributions)

  ⇒ Inclure de **nouvelles fonctionnalités** dans le système d'init

  ⇒ …

Principe

- **Parallélisation des processus** par ouverture de **sockets** : **systemd** peut lancer en parallèle deux processus, même s'ils sont interdépendants.
- Pour chaque processus à lancer, **systemd** ouvre **une socket** unix.
- Les processus dépendant d'un processus donné vont se connecter à la socket correspondante, alors que le service réel n'est pas encore forcément actif.
- Lorsque le processus sera actif, **systemd** le connectera à sa socket et les clients pourront être servis.

Systemd: Organisation

- **systemd** (qui remplace **/sbin/init**) assure (via des fichiers de configuration) l'initialisation du système et la mise en place des services.
- **systemd** introduit la notion d'**unité**. **Une unité représente un fichier de configuration**. Une unité peut être **un service** (*.service), **un target** (*.target), **un montage** (*.mount), **un socket** (*.socket), ...

```
$ systemctl list-units
...
cups.socket      loaded active running   CUPS Printing Service Sockets
home.mount       loaded active mounted   /home
crond.service    loaded active running   Command Scheduler
sshd.service     loaded active running   OpenSSH server daemon
...
```

Démarrage et arrêt du système Linux

Gestionnaire de service Systemd

- Le répertoire **/usr/lib/systemd/system**, contient des fichiers de configuration. Chaque fichier décrit **une unité** et a un nom de la forme sshd.service, graphical.target, cups.socket, etc, suivant son type.

```
$ ls -l /usr/lib/systemd/system/cups*
-r--r--r--. 1 root root 126 20 nov.  16:07 /usr/lib/systemd/system/cups.path
-r--r--r--. 1 root root 198 20 nov.  16:07 /usr/lib/systemd/system/cups.service
-r--r--r--. 1 root root 131 20 nov.  16:07 /usr/lib/systemd/system/cups.socket
```

- Un autre exemple pour **le serveur ssh** :

```
$ rpm -ql openssh-server
...
/usr/lib/systemd/system/sshd-keygen.service
/usr/lib/systemd/system/sshd.service
/usr/lib/systemd/system/sshd.socket
...
```

Systemd: différents types d'unités

- **service :** lancement et administration de processus
- **socket :** lancement d'une socket de type Unix ou réseau, associée à un service.
- **target :** groupe fonctionnel de services et sockets. Généralise la notion de niveau d'éxécution.
- **mount :** gestion des montages, crées à la volée après lecture de /etc/fstab.
- **automount :** auto-montage par autofs.
- **Snapshots:** sauvegarder l'état actuel du système et de le restaurer ultérieurement.

Systemd: Fichiers de configuration

- Le répertoire **/usr/lib/systemd/system** est destiné aux unités système par défaut fournies par le système d'exploitation et les paquets installés,

- **/etc/systemd/system** est destiné aux unités système personnalisées ou modifiées par l'administrateur.

Systemd: commandes systemctl

```
# liste les unités
$ sudo systemctl list-units

# liste les services
$ sudo systemctl list-units --type=service
$ sudo systemctl --state=running

# contrôle d'un service
$ sudo systemctl start|stop|relaod|status|enable|disable <service>

# visualise le contenu d'un service
$ sudo systemctl cat <service>
```

Systemd: Cibles

- Sur les nouveaux systèmes, le concept des niveaux d'exécution a été remplacé par **les cibles (targets) Systemd.**
  - ⇒ Les cibles sont simplement des ensembles logiques d'unités.
  - ⇒ Il s'agit d'un type d'unité spécial **systemd** avec l'extension de fichier **.target**.
  - ⇒ Une cible **systemd** définit **l'état** dans lequel un système doit se trouver, ainsi que **les processus et services qui doivent être lancés** pour entrer dans cet état.

Niveaux d'exécution Systemd et init

SystemVinit Runlevel	Systemd Target	Notes
0	runlevel0.target, poweroff.target	Arrête le système
1, s, single	runlevel1.target, rescue.target	Mode single user.
2, 4	runlevel2.target, runlevel4.target, multi-user.target	Mode défini par l'utilisateur, identique au 3 par défaut.
3	runlevel3.target, multi-user.target	Multi-utilisateur, non grap
5	runlevel5.target, graphical.target	Multi-utilisateur, en mode
6	runlevel6.target, reboot.target	Redémarre

Les cibles (targets)

- Une cible (target) est un groupe fonctionnel d'unités.
- La cible **graphical** lance une interface graphique (**runlevel 5**).

/usr/lib/systemd/system/graphical.target

```
[Unit]
Description=Graphical Interface
Documentation=man :systemd.special(7)
Requires=multi-user.target
After=multi-user.target
Conflicts=rescue.target
AllowIsolate=yes

[Install]
Alias=default.target
```

Les cibles (targets)

- Une cible (target) est un groupe fonctionnel d'unités.
- La cible **graphical** lance une interface graphique (**runlevel 5**).

/usr/lib/systemd/system/graphical.target

```
[Unit]
Description=Graphical Interface
Documentation=man :systemd.special(7)
Requires=multi-user.target
After=multi-user.target
Conflicts=rescue.target
AllowIsolate=yes

[Install]
Alias=default.target
```

graphical.target requiert la cible **multi-user.target** et démarrera après que multi-user ait achevé son démarrage

Les cibles (targets)

- Une cible (target) est un groupe fonctionnel d'unités.
- La cible **graphical** lance une interface graphique (**runlevel 5**).

/usr/lib/systemd/system/graphical.target

```
[Unit]
Description=Graphical Interface
Documentation=man :systemd.special(7)
Requires=multi-user.target
After=multi-user.target
Conflicts=rescue.target
AllowIsolate=yes

[Install]
Alias=default.target
```

graphical.target ne peut pas être activée en même temps que la cible **rescue.target**

Les cibles (targets)

- Une cible (target) est un groupe fonctionnel d'unités.
- La cible **graphical** lance une interface graphique (**runlevel 5**).

/usr/lib/systemd/system/graphical.target

```
[Unit]
Description=Graphical Interface
Documentation=man :systemd.special(7)
Requires=multi-user.target
After=multi-user.target
Conflicts=rescue.target
AllowIsolate=yes

[Install]
Alias=default.target
```

La commande **initctl isolate graphical.target** passera la machine en mode graphique (le **init 5** du bon vieux temps)

Démarrage et arrêt du système Linux

System V init: résumé

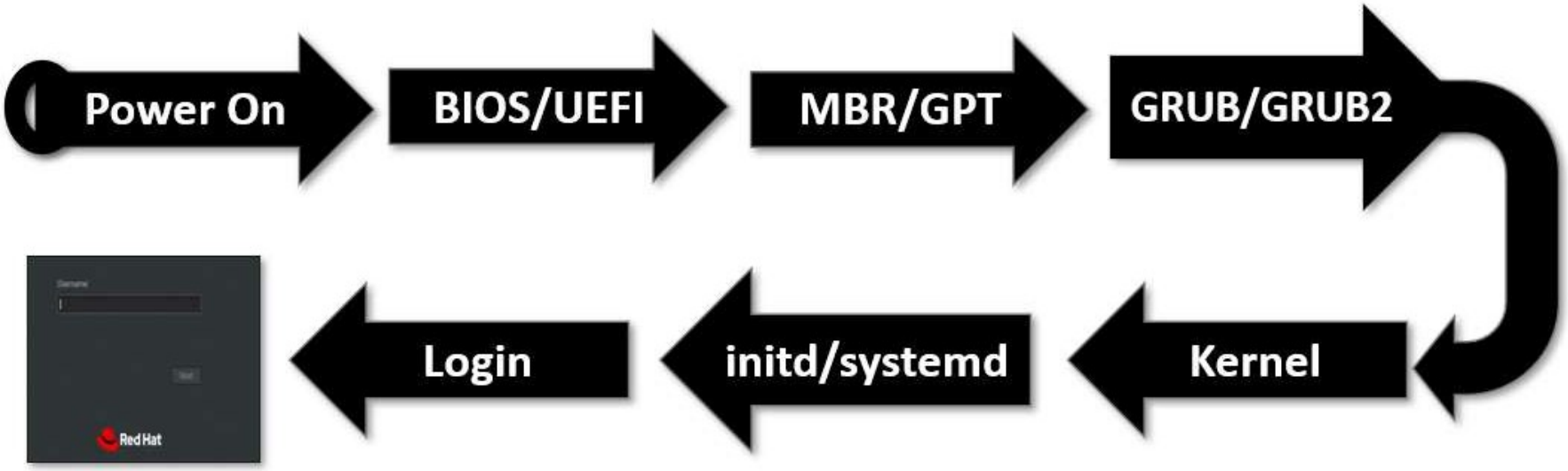
- Système basé sur des **niveaux d'exécution** et une **série de scripts** lancés dans un ordre défini.
  - ⇒ **/etc/inittab** défini le niveau d'exécution
  - ⇒ **/etc/init.d/** contient les scripts écrit la plupart du temps en bash
  - ⇒ **/etc/rd[0-6].d/** contient des liens vers les scripts dont le nom donne l'ordre d'exécution.
- Le système souffrait de **plusieurs défauts** mais avait l'immense avantage d'être **simple et facile** à prendre en main

Systemd: résumé

- Les niveaux d'exécution SystemV sont remplacés par des target unit :
  - ⇒ **poweroff.target:** arrêt du système(runlevel 0)
  - ⇒ **rescue.target:** mode de secours (runlevel 1)
  - ⇒ **multi-user.target:** mode multi-utilisateur sans serveur graphique(runlevel 3)
  - ⇒ **graphical.target:** mode multi-utilisateur avec connexion graphique(runlevel 5)
  - ⇒ **reboot.target:** redémarrage du système (runlevel 6)

Démarrage et arrêt du système Linux

Synthèse



Démarrage et arrêt du système Linux

Arrêt du système (1)

- Les opérations de maintenance, diagnostics, modifications de logiciels, ajouts et retraits de matériel, taches administratives, coupures électriques, … nécessitent parfois **l'arrêt du système**.
- Cet arrêt peut être **planifié**, **périodique** ou **impromptu**
- Tous les systèmes Unix doivent être mis hors service correctement en utilisant les **commandes adéquates.**
- Ceci garantit **l'intégrité du disque** et la terminaison propre des différents services du système.

Démarrage et arrêt du système Linux

Arrêt du système (2)

- **Arrêt programmé** du système :
  - ⇒ utilisateurs prévenus de l'arrêt ;
  - ⇒ applications arrêtées proprement ;
  - ⇒ intégrité des systèmes de fichiers assurée ;
  - ⇒ sessions utilisateurs stoppées.
- **En fonction des options**, le système :
  - ⇒ passe en mode mono-utilisateur ;
  - ⇒ est arrêté ;
  - ⇒ est redémarré.
- **Commandes de mise hors service** :
  - ⇒ init ;
  - ⇒ shutdown ;
  - ⇒ halt ;
  - ⇒ reboot

Démarrage et arrêt du système Linux

Arrêt du système (3)

- Sur les version récentes de **RHEL**, l'utilitaire **systemctl** remplace un certain nombre de commandes de gestion de l'alimentation utilisées dans des versions précédentes du système.
- Les commandes répertoriées dans le tableau ci-après sont **toujours disponibles** sur le système pour des raisons de compatibilité, mais il est conseillé d'utiliser **systemctl** lorsque possible.

Ancienne cmd	Nouvelle cmd	Description
Halt	systemctl halt	Arrête le système.
Poweroff	systemctl poweroff	Met le système hors-tension.
Reboot	systemctl reboot	Redémarre le système.
pm-suspend	systemctl suspend	Suspend le système.
pm-hibernate	systemctl hibernate	Met le système en hibernation.
pm-suspend-hybrid	systemctl hybrid-sleep	Met en hibernation et suspend le système

Démarrage et arrêt du système Linux

Arrêt du système (4)

- Pour **arrêter le système** et **mettre la machine hors-tension** (envoie d'un signal ACPI qui produit l'extinction directe de l'alimentation) :

```
root@hostname# systemctl poweroff
```

- Pour **arrêter** et interrompre le système **sans mettre la machine hors-tension :**

```
root@hostname# systemctl halt
```

halt demande au matériel d'arrêter toutes les fonctions du CPU, mais le laisse sous tension. Vous pouvez l'utiliser pour amener le système dans un état où vous pouvez effectuer une maintenance de bas niveau.

- Pour **redémarrer le système** :

```
root@hostname# systemctl reboot
```

- Par défaut, l'exécution de l'une de ces commandes amène **Systemd** à envoyer **un message informatif** à tous les utilisateurs actuellement connectés au système.
- Pour empêcher **Systemd** d'envoyer ce message: Ajouter l'option **--no-wall**

Démarrage et arrêt du système Linux

Arrêt du système (5): utilisation de la commande shutdown

- La commande **shutdown** joue un rôle essentiel lorsque vous souhaitez **arrêter** ou **redémarrer** Linux via le terminal. Vous pouvez lui adjoindre **une option** suivie par une heure et un message.

```
root@hostname# shutdown [OPTION] [HEURE] [MESSAGE]
```

- **Heure:** soit une heure fixe hh:mm, soit un délai d'attente en minute +mm.
- **Message:** Message à envoyer à tous les utilisateurs.
- **Option :**

Options	Commentaires
-t sec	Attendre sec entre le message d'avertissement et le signal de fin aux processus
-r	Redémarrer la machine après l'arrêt du système
-h	Arrêter la machine après l'arrêt du système
-P	Éteindre l'alimentation
-f	Ne pas effectuer de fsck en cas de redémarrage
-F	Forcer l'utilisation de fsck en cas de redémarrage
-c	Annuler un redémarrage en cours

Démarrage et arrêt du système Linux

Suspendre le système

- Pour suspendre le système :

```
root@hostname# systemctl suspend
```

- Cette commande enregistre l'état du système dans **la RAM** et, à l'exception du module RAM, **met hors-tension la plupart des périphériques dans la machine.**
- Lorsque vous rallumez la machine, le système **restaure son état à partir la RAM sans avoir à redémarrer.**
- Comme l'état du système est enregistré dans la RAM et non sur le disque dur, restaurer le système à partir du mode de suspension est bien **plus rapide** que de le restaurer suite à une hibernation.
- En revanche, un état de système en suspension est également **vulnérable aux pannes d'électricité.**

.

Démarrage et arrêt du système Linux

Hibernation du système

- Pour hiberner le système :

```
root@hostname# systemctl hibernate
```

- Cette commande **enregistre l'état du système** sur le disque dur et met la machine hors-tension.

- Lorsque vous rallumez la machine, le système restaure son état à partir des données enregistrées sans avoir à démarrer à nouveau.

- Comme **l'état du système est enregistré sur le disque dur et non sur la RAM**, la machine **n'a pas besoin de maintenir une alimentation électrique sur le module RAM.**

- En revanche, restaurer le système suite à **une hibernation** est bien **plus lent** que de le restaurer suite au mode de suspension.

Démarrage et arrêt du système Linux

Hiberner et suspendre le système

- Ici, l'état de la machine est sauvegardé dans le swap, mais le système ne s'éteint pas. Au lieu de cela, le PC est suspendu à la RAM.
- La batterie n'est pas utilisée et vous pouvez reprendre le système en toute sécurité à partir du disque et poursuivre votre travail.
- Cette méthode est beaucoup plus lente que la suspension vers la RAM.

```
root@hostname# systemctl hybrid-sleep
```

Démarrage et arrêt du système Linux

Désactiver la suspension et l'hibernation du système

- Pour empêcher votre système Linux de se suspendre ou de se mettre en hibernation, vous devez désactiver les cibles systemd suivantes :

```
root@hostname# systemctl mask sleep.target
suspend.target hibernate.target hybrid-sleep.target
```

- Vérifiez si les changements ont été effectués en utilisant la commande :

```
root@hostname# systemctl status sleep.target
suspend.target hibernate.target hybrid-sleep.target
```

- Pour réactiver les modes de suspension et d'hibernation, exécutez la commande :

```
root@hostname# systemctl unmask sleep.target
suspend.target hibernate.target hybrid-sleep.target
```

Gestion des services

Contexte

Situation:

Vous recevez une alerte : le serveur web de l'entreprise est inaccessible

```
# 1. Diagnostic rapide
$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy
server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vend
or preset: enabled)
   Active: failed (Result: exit-code) since Tue 2024-01-16 08:25:12
CET; 5min ago
     Docs: man:nginx(8)
  Process: 1234 ExecStart=/usr/sbin/nginx (code=exited, status=1/FA
ILURE)
```

Gestion des Services

Gestion des Unités de Service systemd

- Chaque **service** ou **unité** géré par **systemd** est décrit par un fichier de configuration spécifique.
- Ces fichiers contiennent des informations essentielles sur le service, telles que son nom, son exécutable, ses dépendances, etc.
- **Extension: .service**
- **Emplacements : /lib/systemd/system/, /etc/systemd/system/**

```
[Unit]
Description=Mon Service Web
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/mon-service
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

- **[Unit] :** Cette section contient les métadonnées et les conditions de démarrage du service.
- **[Service] :** Cette section précise comment exécuter et gérer le service.
- **[Install] :** Cette section configure l'intégration du service dans le système.

Gestion de Service

États d'un Service

Un service peut se trouver dans l'un des états suivants :

État du service	Description
Actif (active)	Le service est en cours d'exécution et fonctionne correctement.
Inactif (inactive)	Le service est arrêté et n'est pas en cours d'exécution.
Échec (failed)	Le service a rencontré un problème et a échoué.
Masqué (masked)	Le service est masqué : il ne peut pas être activé ni démarré.
Alias (alias)	L'unité est un alias vers une autre unité et ne peut pas être démarrée indépendamment.
Statique (static)	L'unité est statique : elle ne peut pas être activée ou désactivée.
Désactivé (disabled)	Le service est désactivé et ne sera pas démarré automatiquement au démarrage.

Gestion de Services

Commandes de Gestion Essentielles (1)

Catégorie	Commande	Description	Usage Typique
État	systemctl **status** service	État détaillé du service	Diagnostic initial
	systemctl **is-active** service	Vérifie si le service tourne	Scripts automation
	systemctl **is-enabled** service	Vérifie activation au boot	Audit configuration
Contrôle	systemctl **start** service	Démarre le service	Après configuration
	systemctl **stop** service	Arrête le service	Maintenance
	systemctl **restart** service	Redémarre complètement	Après modification majeure
	systemctl **reload** service	Recharge configuration	Changements mineurs

Gestion de Services

Commandes de Gestion Essentielles (2)

Catégorie	Commande	Description	Usage Typique
Démarrage Auto	systemctl **enable** service	Active au boot	Service permanent
	systemctl **disable** service	Désactive au boot	Service manuel seulement
	systemctl **mask** service	Bloque tout démarrage	Service conflictuel
	systemctl **unmask** service	Lève le blocage	Rétablissement
Dépannage	systemctl **daemon-reload**	Recharge config systemd	Après edit unit file
	systemctl **reset-failed**	Réinitialise état échec	Nettoyage après fix
	journalctl -u service -f	Logs temps réel	Debug en direct
Analyse	systemctl **list-unit-files**	Liste tous les services	Audit complet
	systemd-analyze blame	Temps de démarrage	Optimisation boot
	systemctl **show** service	Propriétés détaillées	Investigation avancée

Dépannage et Journalisation

Contexte

Situation: Il est 14h30, un vendredi après-midi...

```
# Votre téléphone sonne - équipe support
"Le serveur principal est inaccessible !
Les utilisateurs ne peuvent plus se connecter.
Les commandes en ligne sont bloquées.
C'EST LA CRISE !"
```

Votre mission : Diagnostiquer et résoudre la panne en 15 minutes maximum !

- **Comment investiguer sans redémarrer ?**
- **Comment trouver La cause dans des milliers de logs ?**

Dépannage et Journalisation

La Solution : journalctl

Commande:

```
# journalctl
```

Objectif :

- Voir ce qui s'est passé à la seconde
- Isoler le service problématique
- Remonter le temps dans les logs
- Identifier les erreurs

Mission : Trouver la cause en moins de 5 minutes !

```
# 1. Connexion SSH d'urgence
ssh admin@serveur-critique

# 2. Quick health check
systemctl status critical-service

# 3. Investigation avec journalctl...
```

Dépannage et Journalisation

La Solution : journalctl

Hypothèses :

- Le problème a commencé il y a 30 minutes
- Un service spécifique est en cause
- Les erreurs doivent être visibles

Avec journalctl :

```
# Filtrer par période et priorité
journalctl --since "30 minutes ago" -p err..alert
```

Résultat de l'investigation :

```
# Exemple de sortie
14:05:00 database.service: Failed to connect to primary
node
14:05:01 webapp.service: Database connection timeout
14:05:02 api.service: Health check failed
```

Cause identifiée: Perte de connexion à la base de données !

Dépannage et Journalisation

La Solution : journalctl

Action corrective :

```
# Redémarrer le service défaillant
sudo systemctl restart database.service

# Vérification en temps réel
journalctl -u database.service -f
```

Résultat : Service rétabli en 2 minutes !

Dépannage et Journalisation

La Solution : journalctl

Commandes Fondamentales de journalctl

Catégorie	Commande	Description	Cas d'Usage	Ex. de Sortie
Base	journalctl	Affiche tous les logs disponibles	Investigation complète, vue d'ensemble	Tous les logs système depuis le début
Temps réel	journalctl -f	Suit les nouveaux logs en direct (follow)	Surveillance active, debugging en direct	Nouveaux messages en temps réel
Boot/ Démarrage	journalctl -b	Logs du démarrage actuel	Problèmes au boot, analyse du cycle actuel	Logs depuis le dernier démarrage
	journalctl -b - 1	Logs du démarrage précédent	Comparaison entre boots, incidents passés	Logs du boot d'avant
	journalctl --list-boots	Liste tous les boots disponibles		

La Solution : journalctl

Commandes Fondamentales de journalctl

Catégorie	Commande	Description	Cas d'Usage	Exemple de Sortie
Service	**journalctl** -u <service>	Logs d'un service spécifique	Debug service particulier, monitoring appli	journalctl -u nginx
Priorité	**journalctl** -p err	Logs de priorité erreur et supérieure	Recherche problèmes critiques	Seulement erreurs système
	journalctl -p warning..emerg	Plage de priorités	Filtrage personnalisé par sévérité	Erreurs + alertes + critiques
Temporel Simple	**journalctl** --since "1 hour ago"	Logs depuis 1 heure	Incident récent, investigation ciblée	Logs de la dernière heure
	journalctl --since "yesterday"	Logs depuis hier	Analyse quotidienne, reporting	Logs depuis minuit

Dépannage et Journalisation

La Solution : journalctl

Commandes Fondamentales de journalctl

Catégorie	Commande	Description	Cas d'Usage	Exemple de Sortie
Temporel Avancé	journalctl --since "09:00" --until "10:00"	Plage horaire précise	Investigation d'incident horodaté	Logs entre 9h et 10h
Limitation	journalctl -n 50	50 dernières lignes seulement	Éviter l'overload, focus sur récent	50 messages les plus récents
Format Sortie	journalctl -o verbose	Format détaillé avec métadonnées	Debug approfondi, analyse complète	Timestamp, PID, UID, etc.

Dépannage et Journalisation

La Solution : journalctl

Les combinaisons indispensables

Usage	Commande	Résultat
Service au démarrage	journalctl -u nginx -b	Logs Nginx depuis boot
Surveillance temps réel	journalctl -u ssh -f	Connexions SSH en direct
Erreurs récentes	journalctl -p err --since "30 min ago"	Incidents dernières 30min
Logs limités	journalctl -u mysql -n 50	50 derniers logs seulement
Crash précédent	journalctl -b -1 -p err	Diagnostiquer plantage
Investigation ciblée	journalctl -u apache2 -p err -n 20	Erreurs récentes Apache
Rapport quotidien	journalctl -p err..alert --since "today"	Problèmes de la journée
Debug complet	journalctl -u service -o verbose	Métadonnées détaillées

Priorités : emerg(0) alert(1) crit(2) err(3) warning(4) info(6) debug(7)